

ITAI-1371 - Intro to Machine Learning
Reflective Journal

Camila Ferreira da Silva

This lab was an opportunity to explore several tools commonly used in machine learning, including Google Colab, pandas, NumPy, matplotlib, and GitHub. Although I followed the notebook and successfully executed the cells, I realized that running existing code is different from fully understanding it or being able to write it independently. The lab introduced me to the purpose of these tools, but I still need more practice before I can use them without guidance.

One of my main challenges was understanding the code. I could recognize some elements, such as variables, functions, and commands used to display results, but I did not always understand why each line was necessary. The sections involving pandas and NumPy showed me how Python can organize data and calculate statistical values. However, I would not yet be able to recreate all these operations independently. This experience helped me identify an important area for improvement: instead of focusing only on whether the code runs successfully, I need to examine each line and understand how it contributes to the final result.

Interpreting the scatter plot was also challenging at first. Looking at the graph, I needed to connect the colors and positions of the points with the three iris species and their sepal measurements. After examining it more carefully, I noticed that setosa was more clearly separated from the other species, while versicolor and virginica overlapped in some areas. This helped me understand that a graph is not only a visual addition to a project. It can reveal patterns and relationships that may be difficult to recognize by looking only at rows of numerical data.

My main insight from this lab was that the different tools work together. Pandas organizes the dataset into a DataFrame, NumPy performs numerical calculations, and matplotlib turns the data into a visual representation. Google Colab provides an environment where the code, explanations, outputs, and graphs can appear in the same document. GitHub then provides a way to store the files, track changes, and share the project. Before completing this lab, I thought of these tools as separate technologies. Now I understand that each one supports a different part of the machine learning workflow.

This lab did not make me feel that I have mastered these tools yet. Instead, it gave me an overview of what they can do and helped me recognize what I need to continue studying. My next step is to practice writing smaller pieces of code independently and predicting the output before running each cell. I would also like to experiment with changing the features used in the visualization to see how the graph changes. Overall, the lab helped me become more familiar with the machine learning environment and gave me a clearer foundation for future assignments.

Danh Nguyen

Analysis and Reflection

- Analysis of the Iris Dataset

The Iris dataset contains 150 samples, four features, and three classes. The features are sepal length, sepal width, petal length, and petal width. The three classes are setosa, versicolor, and virginica.

- Key Findings from the Visualization

The scatter plot compares sepal length and sepal width for the three species. Setosa is more separated from the other two species. Versicolor and virginica have more overlap in the plot. This shows that sepal measurements can help identify differences between the species, but they do not completely separate all three classes.

The dataset also contains 50 samples for each species. The basic statistical analysis provides the mean measurements for each species. These results make it easier to compare the measurements between the three groups.

- Questions for Further Investigation

Two questions that could be explored further are:

- How would the results change if petal length and petal width were used in the visualization?
- Which machine learning algorithm would classify the three iris species most accurately?
- Reflection

This lab helped me understand how Google Colab can be used to run Python code and work with a dataset. I learned how Pandas, NumPy, Matplotlib, and Scikit-learn can be used for data analysis and visualization. I also learned how a simple graph can help identify patterns in a dataset.

Great Inioluwa Olayiwola

By completing this exercise, I learned how seamlessly pandas DataFrames structure raw arrays into labelled tables and how NumPy simplifies quick array computations like mean and standard deviation. Additionally, visualizing data using Matplotlib revealed natural groupings and feature overlaps that were not obvious from looking at numeric tables alone. Before this lab, my mental model for structured data was strictly spreadsheet oriented. Coming from standard Excel tables, I initially found pandas DataFrames slightly nonintuitive specifically when transitioning from implicit row numbers to explicit index labels and handling multidimensional arrays. However, working through the indexing exercises triggered a key breakthrough: pandas indexing isn't just a row counter, but a high-performance hash-map key system that allows for relational alignment across complex datasets.

The contrast between native Python lists and vectorized NumPy arrays was another major point of growth. Previously, I viewed iteration through `'for'` loops as the default way to process sequential data. Watching vectorized operations compute mathematical metrics like mean and standard deviation across entire array columns instantaneously forced me to rethink computational efficiency. Seeing vectorization in action transformed NumPy from an abstract linear algebra library into an essential engine for scalable machine learning workflows.

My primary struggle during this lab was not in writing the Python code itself, but in managing plot boundaries, margins, and rendering layouts during document export. When generating the sepal dimension scatter plots and class distribution bar charts using Matplotlib, key elements like axis labels and visual legends were initially clipped or truncated during the PDF conversion process.

Initially, I tried manually tweaking figure dimensions, but this proved imprecise across different page aspect ratios. Furthermore, troubleshooting the Jupyter-to-PDF export pipeline highlighted a crucial lesson in software deployment: code execution is only half the battle; output presentation and static asset rendering require equal technical attention.

Karima Sadruddin

Module 2: Tools Used in Machine Learning This lab changed my understanding of machine-learning tools because I could see how several tools work together as one workflow. Before working with the Iris dataset, pandas, NumPy, and Matplotlib could seem like separate programming topics. Using them in sequence made the connection clearer: pandas organized the data, NumPy performed numerical calculations, and Matplotlib turned numerical relationships into a visual pattern. The workflow showed me that learning a tool is not only about memorizing commands; it is about understanding what problem each tool helps solve.

One important insight came from comparing the table of Iris measurements with the scatter plot. The numerical output contains many values, but the visualization makes grouping and overlap easier to recognize. The plot showed a clear separation for setosa while versicolor and virginica overlapped more. That helped me understand why visualization is useful before building a machine-learning model: it can reveal structure, overlap, and questions that may not be obvious from raw numbers alone.

The statistical calculations also helped connect programming with concepts I already know from mathematics. Calculating the mean and standard deviation with NumPy was more meaningful when I could relate the results to the actual sepal-length measurements. I learned that a mean gives a central value, while standard deviation gives information about how spread out the observations are. This made the code feel less like a formula to memorize and more like a practical way to summarize data.

A challenge in this lab was keeping track of the relationship between the dataset structure and the code. For example, the Iris dataset begins with numeric target values, while the DataFrame uses readable species names. Following that transformation helped me see why data preparation matters: readable and well-organized data makes later analysis easier and less error-prone. I also learned to pay attention to verification statements and output messages because they provide quick evidence that a calculation or task worked as intended.

The lab also changed how I think about reproducibility. Running the notebook from the beginning is important because later cells depend on variables created earlier. Re-running the cells helped reinforce that a successful notebook should not depend on hidden or accidental state. This is especially important in machine learning, where another person should be able to follow the same workflow and understand how the results were produced.

For further learning, I would like to investigate whether petal measurements separate the three Iris species more clearly than sepal measurements. I would also like to experiment with additional visualizations and standardization. These questions connect the basic tools from this lab to the larger machine-learning workflow I will use in later modules.

Overall reflection: The most useful lesson was learning how the tools complement one another. Instead of viewing Python libraries as isolated requirements, I now see them as parts of a repeatable data-analysis process: load and organize data, calculate useful summaries, visualize patterns, verify results, and document the work clearly.

Quan Thach

We talked about where we even write and run our code. The lab looks at interactive environments like Jupyter Notebook, Google Colab, and Visual Studio Code. If you want the easiest route without messing up your local computer paths or environment variables, Google Colab is clutch because everything is pre-installed and you don't have to install local libraries. On the flip side, if you're using VS Code or local notebooks, you have to deal with regular Python installs, run terminal commands like `pip install`, and set up specific extensions like the Python and Jupyter extensions. Plus, we use GitHub to store and share our code projects and Markdown to write clean project documentation. It sounds like a lot of tools, but you pick up the rhythm pretty fast.

Next up, the Python libraries we imported are basically our main toolkit for this stuff. We use standard aliases so everyone writes cleaner code. Pandas is basically Excel on steroids for tabular data manipulation. We use it to load dataset features into a `DataFrame`, add a target species column, and look at the top rows using `.head()` or check the data structure with `.info()`. NumPy (`np`) helps us crunch numerical arrays, and Matplotlib (`plt`) is what we grab when we want to make charts and graphs to visualize stuff. Then there's Scikit-learn (`sklearn`), which is our go-to machine learning library here, specifically using `sklearn.datasets.load_iris` to pull in sample data. We also throw in `warnings.filterwarnings('ignore')` just to hide annoying warning messages so our output looks clean.

Speaking of data, the whole exploration workflow in this lab uses the classic Iris dataset. Instead of just training a model blindly, the point is to programmatically inspect what's inside multi-class classification data. We check if the data loaded right, look at the dimensional shape, which is `(150, 4)`, and pull up the feature names: `['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']`. We also inspect the target classes (`['setosa', 'versicolor', 'virginica']`) and check non-null data types and memory usage. The big takeaway the lab pushes here is that early data visualization isn't optional or just for looks—it's a critical preliminary step to actually build an intuitive understanding of your dataset before you even think about training a model.

Honestly, going through this lab makes the transition from theory to actual coding feel way more grounded. Instead of memorizing abstract math formulas right away, you're literally just seeing how data shapes look in pandas, checking your dimensions, cleaning your output warnings, and prepping visualizations.